BVG

Binary Vector Graphics — Format Specification

BVG contributors

2026-11-06

Contents

1 Abstract **4**

2 Introduction **5**
- 2.1 Motivation . . . 5
- 2.2 Scope . . . 5
- 2.3 Design principles . . . 5

3 Conformance **7**
- 3.1 Reader . . . 7
- 3.2 Writer . . . 7

4 Terminology **9**

5 File Layout **10**

6 Header **11**
- 6.1 Header examples . . . 12

7 Flags **13**

8 Chunk Table **14**
- 8.1 Global layout (12 bytes per record) . . . 14
- 8.2 Per-chunk layout (15 bytes per record) . . . 14
- 8.3 Chunk ordering . . . 15
- 8.4 Unknown chunks . . . 15

9 Codecs **16**
- 9.1 Codec availability . . . 16
- 9.2 Choosing a codec . . . 16

10 RLE **17**
- 10.1 Encoder rules . . . 17
- 10.2 Edge cases . . . 18

11 RLE prefilter **19**
- 11.1 Why prefilter . . . 19
- 11.2 Interaction with codec . . . 19

12 Chunks **20**
12.1 COLR - palette . . . 20
12.2 GRAD - gradients . . . 21
12.2.1 Linear gradient . . . 21
12.2.2 Radial gradient . . . 22
12.2.3 Conic gradient . . . 22
12.2.4 Pattern . . . 22
12.3 IMGS - raster resources . . . 22
12.3.1 Format identifiers . . . 22
12.3.2 Memory and security . . . 23
12.3.3 Missing image . . . 23
12.4 STYL - styles . . . 23
12.4.1 Fill kinds . . . 23
12.4.2 Stroke kinds . . . 24
12.4.3 Cap and join . . . 24
12.4.4 Blend modes . . . 24
12.5 NODE - scene tree . . . 24
12.5.1 Rect . . . 25
12.5.2 Ellipse . . . 25
12.5.3 Line . . . 25
12.5.4 Polyline and polygon . . . 25
12.5.5 Path . . . 25
12.5.6 Depth . . . 26
12.5.7 Groups . . . 26
12.5.8 Def and Use . . . 26
12.5.9 Text . . . 26
12.5.10 Image . . . 26
12.5.11 Transforms . . . 26
12.6 Path commands . . . 27
12.7 FONT - reserved . . . 27
12.8 META - metadata . . . 27
12.9 THMB - thumbnail . . . 27

13 Coordinate System **28**
13.1 Path arcs . . . 28
13.2 Stroke geometry . . . 28

14 Error Model **29**
14.1 When to fail . . . 29
14.2 When to recover . . . 30

15 Change Log **31**

16 Glossary **32**

17 References **33**

Document	BVG - Binary Vector Graphics
Version	1.0.0
Date	2026-11-06
Status	Stable

Chapter 1

Abstract

BVG (Binary Vector Graphics) is a binary container format for 2D vector graphics. It stores geometry, styles, gradients, palettes and a tree of groups in a compact, deterministic, chunk-oriented file. Raster resources (PNG, JPEG, raw RGBA) are supported as optional embedded resources referenced by vector nodes; the drawing itself remains vector.

BVG is designed to be:

- **fast** - flat struct arrays, no XML, no DOM,
- **compact** - palette + indices, `float32`, multiple codecs,
- **deterministic** - no CSS cascade, no `use`, no namespaces,
- **extensible** - chunk-based container, unknown chunks are skipped,
- **self-describing** - every chunk has a 4-byte type and a known binary layout.

The key words MUST, MUST NOT, SHOULD, SHOULD NOT, MAY in this document are to be interpreted as described in RFC 2119.

Chapter 2

Introduction

2.1 Motivation

SVG is the de-facto standard for vector graphics on the web, but it is textual XML. Parsing a 400 KiB SVG with 10 000 paths takes tens of milliseconds and allocates tens of megabytes of DOM. BVG stores the same content as flat struct arrays, which parse in microseconds and allocate exactly the size of the data.

BVG is not a replacement for SVG. SVG remains the interchange format for the web. BVG is an execution and storage format for pipelines that produce, process or render vector graphics at scale: tracers, CAD viewers, game engines, embedded renderers, intermediate representations.

2.2 Scope

BVG v1 defines:

- a binary container (header, chunk table, chunk payloads),
- a set of codecs (none, zlib, zstd, lz4, rle) with per-chunk levels,
- a vector scene model (groups, primitives, paths, text, images),
- styles (fill, stroke, opacity, dash, blend mode),
- gradients (linear, radial, conic, pattern),
- palettes and raster resources,
- rules for reading, writing, skipping unknown chunks and erroring.

BVG v1 does **not** define:

- animation or time,
- fonts (a `FONT` chunk is reserved but empty in v1),
- filters, masks, clip paths (future versions),
- compression dictionaries shared between chunks.

2.3 Design principles

1. **Little-endian everywhere.** No endianness flags.
2. **Fixed-size header.** 32 bytes, no varints.

3. **Chunk-oriented.** Each chunk is independent and skippable.
4. **Per-chunk codec and level.** Different chunks have different entropy; a single global codec is suboptimal.
5. **RLE prefilter is optional per chunk.** It helps on flat arrays and hurts on already-compressed data.
6. **No CSS, no `use`, no namespaces.** What is in the file is what is drawn.
7. **Unknown chunks are preserved by readers that re-write.** A tool that edits a BVG file MUST keep unknown chunks intact.

Chapter 3

Conformance

3.1 Reader

A conforming reader MUST:

1. Read the 32-byte header.
2. Verify `magic == b'BVG\x00'` - all four bytes.
3. Read `version` from offset 4; reject if greater than the implemented version.
4. Read the chunk table, both layouts.
5. Decode all codecs 0-4.
6. Apply the RLE prefilter when `CHUNK_PRE_RLE` is set.
7. Read chunks `COLR`, `GRAD`, `IMGS`, `STYL`, `NODE`, `META`.
8. Skip chunks with unknown type without error.
9. Reject files whose `total_uncompressed` does not match the sum of decoded chunk sizes (SHOULD).

A conforming reader MAY:

- render the scene to a raster surface,
- export the vector content to another format,
- use a `THMB` preview.

3.2 Writer

A conforming writer MUST:

1. Emit `magic = b'BVG\x00'` and `version = 1`.
2. Emit a chunk table with correct offsets and sizes.
3. Use only codecs 0-4.
4. Set `total_uncompressed` to the sum of raw chunk sizes.
5. Write chunks in the order they appear in the file, preserving unknown chunks read from a source file.
6. Set `FLAG_PERCHUNK` if any chunk uses a codec or level different from the header defaults.
7. Emit a valid `NODE` chunk (possibly empty).

A conforming writer SHOULD:

- produce a file that round-trips: read it back, get the same chunks,
- avoid writing `zstd` or `lz4` if the target reader may not support them.

Chapter 4

Terminology

- **Chunk** - a named block of data (`COLR`, `GRAD`, …).
- **Chunk type** - 4-byte ASCII identifier.
- **Chunk payload** - uncompressed bytes of the chunk.
- **Compressed size** - size of the chunk payload after the main codec.
- **Raw size** - size of the chunk payload before any compression.
- **Codec** - algorithm that maps payload to compressed bytes and back.
- **Level** - codec-specific tuning parameter.
- **RLE prefilter** - optional `rle` encode applied before the main codec.
- **Scene tree** - the `NODE` chunk, a pre-order list of nodes.
- **Style** - fill, stroke, opacity, dash, blend mode.
- **Palette** - the `COLR` chunk, a list of RGBA colors.
- **Gradient** - a `GRAD` entry, linear / radial / conic / pattern.
- **Resource** - an entry in `IMGS`, referenced by index.
- **Pre-order** - tree traversal: parent first, then children left to right.
- **Affine matrix** - 2x3 matrix describing translation, rotation, scale, shear.

Chapter 5

File Layout

A BVG file is:

```
+------------------------------+
| Header (32 bytes)            |
+------------------------------+
| Chunk table (N x 12 or 15)   |
+------------------------------+
| Chunk data area              |
|   chunk 0 (compressed)       |
|   chunk 1 (compressed)       |
|   ...                        |
+------------------------------+
```

The chunk data area starts immediately after the chunk table. Each chunk's `offset` is relative to the start of the chunk data area, not to the start of the file.

Chapter 6

Header

The header is exactly **32 bytes**, little-endian.

Offset	Size	Field	Value
0	4	`magic`	`b'BVG\x00'` - bytes `42 56 47 00`
4	2	`version`	`1`
6	2	`flags`	see Section “Flags”
8	4	`width`	canvas width in pixels
12	4	`height`	canvas height in pixels
16	4	`chunk_count`	number of chunks
20	1	`codec_id`	default codec
21	1	`level`	default level
22	2	reserved	MUST be `0`
24	8	`total_uncompressed`	sum of raw chunk sizes

Magic. The first four bytes are `B`, `V`, `G`, `0x00`. Readers MUST compare all four bytes. The magic does not encode the format version; the version is a separate field at offset 4.

Version. A 16-bit unsigned integer. This document specifies version `1`. Readers MUST reject files whose `version` is greater than the highest version they implement. Readers SHOULD reject files whose `version` is `0`.

Width / height. Canvas dimensions in pixels. They are informational; renderers MAY use them as the default viewport.

Chunk count. The number of chunk table records.

Default codec and level. Used when `FLAG_PERCHUNK` is not set. When `FLAG_PERCHUNK` is set, these fields are ignored.

Reserved. MUST be zero. Readers SHOULD reject files where reserved bytes are non-zero.

Total uncompressed. Sum of raw chunk sizes. Used by `info`-style tools without decompressing chunks. Readers MAY verify it.

6.1 Header examples

Empty 1x1 file:

```
42 56 47 00   magic = 'BVG\0'
01 00         version = 1
00 00         flags = 0
01 00 00 00   width = 1
01 00 00 00   height = 1
00 00 00 00   chunk_count = 0
00            codec_id = 0
00            level = 0
00 00         reserved
00 00 00 00 00 00 00 00   total_uncompressed = 0
```

Chapter 7

Flags

Bit	Name	Meaning
0	`FLAG_PERCHUNK`	chunk table is 15 bytes per record
1	`FLAG_U32XY`	coordinates are `float32` (currently always on)
2	`FLAG_HAS_THUMB`	file contains a `THMB` chunk
3	`FLAG_HAS_META`	file contains a `META` chunk
4..15	reserved	MUST be zero

`FLAG_U32XY` is reserved for a future version where coordinates may be `float16`. In version 1, coordinates are always `float32`, and this flag is set by writers.

Chapter 8

Chunk Table

The chunk table follows the header and contains `chunk_count` records. Two layouts are defined. The layout is chosen by `FLAG_PERCHUNK`.

8.1 Global layout (12 bytes per record)

Used when `FLAG_PERCHUNK` is **not** set. All chunks share the codec and level from the header.

Offset	Size	Field
0	4	`type` - 4-byte ASCII
4	4	`offset` - relative to chunk data area
8	4	`size` - compressed size

8.2 Per-chunk layout (15 bytes per record)

Used when `FLAG_PERCHUNK` **is** set. Each chunk has its own codec, level and flags.

Offset	Size	Field
0	4	`type` - 4-byte ASCII
4	4	`offset` - relative to chunk data area
8	4	`size` - compressed size
12	1	`codec_id`
13	1	`level`
14	1	`flags`

Chunk flags:

Bit	Name	Meaning
0	`CHUNK_PRE_RLE`	RLE prefilter applied before the main codec
1..7	reserved	MUST be zero

8.3 Chunk ordering

The recommended order is:

`COLR -> GRAD -> IMGS -> FONT -> STYL -> NODE -> META -> THMB`

Readers MUST NOT depend on this order, except that `NODE` may reference entries in `COLR`, `GRAD`, `IMGS`, `STYL` - those chunks MUST be read before rendering the scene, regardless of file order.

8.4 Unknown chunks

A reader MUST skip chunks with unknown type without error. A writer that edits a file MUST preserve unknown chunks byte-for-byte, including their position in the chunk table.

Chapter 9

Codecs

ID	Name	Levels	Default	Notes
0	`none`	-	-	no compression
1	`zlib`	1..9	6	DEFLATE with zlib wrapper
2	`zstd`	1..22	3	Zstandard frame
3	`lz4`	0..16	0	LZ4 frame
4	`rle`	-	-	BVG RLE (see Section “RLE”)

Level `0` means “codec default”. A reader MUST support `none`, `zlib` and `rle`. A reader SHOULD support `zstd` and `lz4` if the corresponding library is available. If a reader encounters a codec it does not support, it MUST reject the file with an error that names the codec.

9.1 Codec availability

The `codec_id` field is always present. A file may use `zstd` even if the reader does not have Zstandard support; the reader will fail at decode time with a codec error.

9.2 Choosing a codec

There is no single “best” codec. Writers SHOULD choose by size:

- `none` - for tiny chunks where codec overhead exceeds savings,
- `rle` - for arrays with long runs of equal bytes,
- `zlib` - for generic data, always available,
- `zstd` - for large chunks where ratio matters,
- `lz4` - for large chunks where decode speed matters.

Chapter 10

RLE

BVG RLE is a simple byte-oriented codec. The stream is a sequence of `(header, payload)` pairs. The header byte controls the pair type.

Header bit 7	Meaning	Payload
0	literal	`header + 1` bytes taken from the stream
1	run	`(header & 0x7F) + 3` copies of one byte

Decoding algorithm:

```
while not end of stream:
    h = read_byte()
    if h & 0x80:
        n = (h & 0x7F) + 3
        b = read_byte()
        output n copies of b
    else:
        n = h + 1
        output next n bytes
```

Encoding example:

```
Input:  00 00 00 00 00
Output: 82 00
```

`0x82` = `0x80 | 0x02`, so `(0x02) + 3 = 5` copies of `0x00`.

Input: `41 42 43` Output: `02 41 42 43` `0x02` = literal of `0x02 + 1 = 3` bytes.

RLE is never applied recursively. The RLE prefilter (next section) uses RLE once, before the main codec.

10.1 Encoder rules

A writer MUST produce a valid stream. It MAY choose literal or run for any input, but SHOULD prefer runs when `run >= 3`. There is no requirement to produce the smallest possible stream.

A reader MUST accept any valid stream, including a stream that uses literals for data that could be runs.

10.2 Edge cases

- Empty input -> empty output.
- Input of length 1 -> literal of 1 byte (`00 <byte>`).
- Runs longer than 130 must be split into multiple run records.
- Literals longer than 128 must be split into multiple literal records.

Chapter 11

RLE prefilter

The RLE prefilter is an optional step applied **before** the main codec. It is enabled per chunk by `CHUNK_PRE_RLE`.

Encoding (writer):

1. If `CHUNK_PRE_RLE` is set and the codec is not `rle`: `payload = rle_encode(payload)`.
2. `compressed = codec_encode(payload)`.

Decoding (reader):

1. `payload = codec_decode(compressed)`.
2. If `CHUNK_PRE_RLE` is set and the codec is not `rle`: `payload = rle_decode(payload)`.

The `rle` codec ignores `CHUNK_PRE_RLE` - applying RLE twice would double-encode.

11.1 Why prefilter

Some chunks have long runs of equal bytes: `COLR` with repeated colors, `STYL` with repeated style records, `NODE` with repeated zero bytes in `float32` coordinates. RLE compresses those runs before zlib/zstd/lz4 see them, which can improve the final ratio.

The prefilter is not always beneficial. On random data it adds 1 byte per 128 literal bytes. Writers SHOULD try both with and without the prefilter and pick the smaller result.

11.2 Interaction with codec

The prefilter is orthogonal to the codec. Any of `none`, `zlib`, `zstd`, `lz4` may be combined with `CHUNK_PRE_RLE`. The `rle` codec itself ignores the flag.

Chapter 12

Chunks

Each chunk has a 4-byte type. This document defines the following types.

Type	Name	Required in v1
`COLR`	Palette	if any style uses a color
`GRAD`	Gradients	if any style uses a gradient
`IMGS`	Raster resources	if any node or gradient references an image
`STYL`	Styles	if any node has a style
`NODE`	Scene tree	always (may be empty)
`FONT`	Fonts	reserved, empty in v1
`META`	Metadata	optional
`THMB`	Thumbnail	optional

All multi-byte fields are little-endian. All coordinates are `float32` unless otherwise specified. All indices are `uint16`, with `0xFFFF` as a sentinel meaning "none".

A chunk is present in the file if and only if it appears in the chunk table. A chunk with `size = 0` is valid and means "empty payload". An absent chunk is equivalent to a zero-count chunk, except where noted (`NODE` is always present, `COLR`/`GRAD`/`IMGS`/`STYL` may be absent if nothing references them).

12.1 `COLR` - palette

A list of RGBA colors, 4 bytes each. Colors are not premultiplied.

```
uint16 count
count x (uint8 R, uint8 G, uint8 B, uint8 A)
```

Example: a single red color.

```
01 00        count = 1
FF 00 00 FF  R=255, G=0, B=0, A=255
```

Total: 6 bytes.

Colors are referenced by index from `STYL` and `GRAD`. An index of `0xFFFF` is a sentinel meaning "none" and MUST NOT be used as a color index. A reader MUST validate that indices used by `STYL` and `GRAD` are less than `count`; out-of-range indices are a chunk error.

The palette is flat and has no ordering semantics. Writers SHOULD deduplicate identical colors. Readers MUST NOT assume colors are unique.

An absent `COLR` chunk is equivalent to an empty palette (`count = 0`). In that case, any style that references a color by index is invalid, and a reader MUST reject the file.

12.2 `GRAD` - gradients

Gradients used by styles and patterns. A gradient is identified by index into this list.

```
uint16 count
count x:
  uint8  kind         // 0 linear, 1 radial, 2 conic, 3 pattern
  uint8  flags        // bit 0 = has_transform
  uint16 stop_count
  stop_count x:
    float32 offset    // 0.0 .. 1.0
    uint16  color_idx // 0xFFFF = inline RGBA
    [uint8 R, uint8 G, uint8 B, uint8 A]  // only if color_idx == 0xFFFF
  // geometry, depends on kind:
  kind 0 (linear) : float32 x1, y1, x2, y2
  kind 1 (radial) : float32 cx, cy, r, fx, fy
  kind 2 (conic)  : float32 cx, cy, start_angle, end_angle
  kind 3 (pattern): uint16 image_idx, float32 x0, y0, x1, y1
  // optional transform (if flags & 1):
  float32 a, b, c, d, e, f
```

Stop offsets are increasing in `[0, 1]`. Stops with `color_idx == 0xFFFF` carry an inline RGBA color; other stops index into `COLR`.

The `transform` is an affine matrix `(a, b, c, d, e, f)` interpreted as:

```
| a  c  e |
| b  d  f |
| 0  0  1 |
```

It applies to the gradient geometry, not to the shape.

`GRAD_PATTERN` references an image from `IMGS`. The rectangle `(x0, y0, x1, y1)` defines the tiling area; the image is scaled to fit that rectangle and tiled across the shape.

12.2.1 Linear gradient

A linear gradient interpolates between stops along the line from `(x1, y1)` to `(x2, y2)`. Points on the line are mapped to offsets `0` and `1`; points outside the segment are clamped.

12.2.2 Radial gradient

A radial gradient interpolates between stops along a circle centered at `(cx, cy)` with radius `r`. The focal point `(fx, fy)` shifts the gradient. If `fx == cx` and `fy == cy`, the gradient is a standard radial gradient.

12.2.3 Conic gradient

A conic gradient interpolates between stops around the center `(cx, cy)` from `start_angle` to `end_angle`. Angles are in radians. Values outside the range are wrapped.

12.2.4 Pattern

A pattern gradient references an image in `IMGS` by index. The image is scaled to fit `(x0, y0, x1, y1)` and tiled across the shape. `flags` controls tiling:

Bit	Meaning
1	repeat in x
2	repeat in y
3	reflect in x (mirror)
4	reflect in y (mirror)

If bits 1 and 2 are both zero, the image is painted once and the rest of the shape is transparent.

12.3 `IMGS` - raster resources

A list of embedded images. Each image is referenced by index from `NODE_IMAGE` or `GRAD_PATTERN`.

```
uint16 count
count x:
  uint8  format       // 0 PNG, 1 JPEG, 2 raw RGBA
  uint16 width
  uint16 height
  uint32 data_len
  uint8[data_len]     // PNG/JPEG bytes, or width*height*4 RGBA bytes
```

For `format == 2` (raw RGBA), `data_len` MUST be `width * height * 4`. Bytes are stored row-major, top-to-bottom, not premultiplied.

`IMGS` is a resource table. It is not rendered by itself; images are referenced by `NODE_IMAGE` or `GRAD_PATTERN`.

12.3.1 Format identifiers

ID	Format	`data_len`
0	PNG	any valid PNG byte stream
1	JPEG	any valid JPEG byte stream
2	raw RGBA	`width * height * 4`

A reader MUST validate `data_len` against `width * height * 4` for `format == 2`. For PNG and JPEG, a reader MUST validate the byte stream before decoding; a corrupted stream is a chunk error.

12.3.2 Memory and security

A reader MUST cap decoded image dimensions to a sane maximum (for example, 16384 x 16384) and cap total decoded size. An image that claims `width * height * 4 > total_uncompressed` is suspicious and MAY be rejected.

12.3.3 Missing image

A `NODE_IMAGE` or `GRAD_PATTERN` that references an `IMGS` index out-of-range is a chunk error. A reader MUST NOT silently skip such a node; it MUST report the error and abort rendering of the scene.

12.4 `STYL` - styles

A list of styles. Each style describes fill and stroke of a node.

```
uint16 count
count x:
  uint8  fill_kind    // 0 none, 1 solid, 2 gradient
  uint16 fill_idx     // color index or gradient index
  uint8  stroke_kind  // 0 none, 1 solid, 2 gradient
  uint16 stroke_idx
  float32 stroke_width
  uint8  cap          // 0 butt, 1 round, 2 square
  uint8  join         // 0 miter, 1 round, 2 bevel
  float32 miter_limit
  uint8  dash_count
  dash_count x float32 dash
  uint8  opacity      // 0..255
  uint8  blend_mode   // 0 normal, 1 multiply, 2 screen, 3 overlay
```

`fill_idx` and `stroke_idx` are indexes into `COLR` or `GRAD` depending on the corresponding `*_kind`. When `kind == 0` (none), the index is ignored and SHOULD be `0`.

`dash` is a list of alternating on/off lengths in user units. Empty list means solid line.

`opacity` is a global alpha multiplier applied to the whole node.

`blend_mode` is a hint; readers MAY ignore it.

12.4.1 Fill kinds

ID	Name	Index into
0	none	-
1	solid	`COLR`
2	gradient	`GRAD`

12.4.2 Stroke kinds

ID	Name	Index into
0	none	-
1	solid	`COLR`
2	gradient	`GRAD`

12.4.3 Cap and join

Cap	Meaning	Join	Meaning
0	butt	0	miter
1	round	1	round
2	square	2	bevel

`miter_limit` is used only when `join == 0` (miter). It is the maximum ratio of miter length to stroke width. Values below 1 are invalid.

12.4.4 Blend modes

ID	Name
0	normal
1	multiply
2	screen
3	overlay

A reader MAY ignore `blend_mode` and draw with `normal`. A reader that supports blend modes MUST implement at least `normal` and MAY implement the rest.

12.5 `NODE` - scene tree

A depth-first pre-order list of nodes. Each node starts with a common header, followed by a type-specific payload.

Common header:

```
uint8  type
uint8  flags        // bit 0 = has_transform
uint16 style_idx    // 0xFFFF = no style
uint16 depth        // 0 = root
[float32 a, b, c, d, e, f]  // only if flags & 1
```

Node types:

Code	Type	Payload
0	Group	`uint16 child_count`
1	Rect	`float32 x, y, w, h`
2	Ellipse	`float32 cx, cy, rx, ry`
3	Line	`float32 x1, y1, x2, y2`
4	Polyline	`uint16 n`, `n x (float32 x, float32 y)`
5	Polygon	`uint16 n`, `n x (float32 x, float32 y)`
6	Path	`uint16 cmd_count`, see "Path commands"
7	Text	`uint16 font_idx, float32 x, y, uint16 len, utf8[len]`
8	Image	`uint16 image_idx, float32 x, y, w, h`
9	Use	`uint16 def_idx, float32 x, y`
10	Def	`uint16 child_count`

Node flags:

Bit	Name	Meaning
0	`NODE_HAS_TRANSFORM`	6 floats `a..f` follow the header
1..7	reserved	MUST be zero

12.5.1 Rect

A rectangle with top-left `(x, y)` and size `(w, h)`. `w` and `h` MUST be non-negative. A zero-size rect is valid and draws nothing.

12.5.2 Ellipse

An ellipse centered at `(cx, cy)` with radii `(rx, ry)`. `rx` and `ry` MUST be non-negative. `rx == ry` is a circle.

12.5.3 Line

A line segment from `(x1, y1)` to `(x2, y2)`. A line has no fill; it is stroked by `stroke_kind` of its style.

12.5.4 Polyline and polygon

An ordered list of points. Polyline is open (no implicit closing edge), polygon is closed. Both use the node's fill and stroke; a polyline with no stroke draws nothing visible.

12.5.5 Path

A sequence of path commands (see "Path commands"). A path may be open or closed; fill uses the non-zero winding rule unless `flags` says otherwise (bit 2 = even-odd fill).

12.5.6 Depth

For a child of a group, `depth = parent.depth + 1`. Roots have `depth = 0`. Readers that use a stack MUST validate that depth values are consistent: a node at depth `d` MUST have a parent at depth `d-1`, except roots at depth `0`.

12.5.7 Groups

A group occupies a contiguous block in pre-order. `child_count` tells how many nodes follow before the next sibling. A group with `child_count = 0` is valid and renders nothing.

Rendering a group means rendering its children in order, with the group's transform (if any) applied to all of them and the group's style (if any) applied to children that do not have their own style.

12.5.8 Def and Use

`Def` is a reusable subtree, not drawn by itself. `Use` references a `Def` by index and places it at `(x, y)`. Defs are indexed in the order they appear in the `NODE` chunk, starting at 0.

A `Def` node's payload is the same as a `Group`: `child_count`. The subtree follows immediately after the `Def`, pre-order.

A `Use` node's payload is `uint16 def_idx, float32 x, y`. The referenced def is rendered at `(x, y)` with the use node's transform (if any) and style (if any).

12.5.9 Text

`font_idx` indexes a `FONT` entry. In version 1, `FONT` is reserved and empty; writers SHOULD use `font_idx = 0` and a style with fill and stroke. Readers MAY render text with a default font.

The `len` field is the number of UTF-8 bytes in the text, not the number of characters. Readers MUST validate `len` against the remaining payload size.

12.5.10 Image

`image_idx` indexes `IMGS`. The rectangle `(x, y, w, h)` defines the placement. The image is scaled to fit; preserving aspect ratio is **not** implied; `w` and `h` are independent.

An image node's style MAY set opacity, but fill and stroke are ignored - an image covers its rectangle.

12.5.11 Transforms

A node with `NODE_HAS_TRANSFORM` has an affine matrix that applies to its geometry and to its children (for groups). A matrix is stored as six floats in the order `a, b, c, d, e, f`, interpreted as:

```
| a  c  e |
| b  d  f |
| 0  0  1 |
```

The matrix composes with the parent's matrix: a child's transform applies first, then the parent's.

12.6 Path commands

Each command starts with a `uint8` opcode:

Op	Name	Payload
0	Move	`float32 x, y`
1	Line	`float32 x, y`
2	Quad	`float32 cx, cy, x, y`
3	Cubic	`float32 c1x, c1y, c2x, c2y, x, y`
4	Arc	`float32 rx, ry, rot; uint8 large_arc, sweep; float32 x, y`
5	Close	-

The path starts with `Move`. `Close` connects the current point to the last `Move`. `Arc` follows the SVG arc parameterization (`large_arc`, `sweep` are 0 or 1).

A path MAY contain multiple subpaths (multiple `Move` commands). Fill applies to all closed subpaths; stroke applies to all subpaths.

Path flags (node-level, `flags` bit 2):

Bit	Meaning
2	even-odd fill rule (default is non-zero)

12.7 `FONT` - reserved

Not used in version 1. The chunk, if present, MUST be empty (count = 0). Readers MUST skip its contents.

12.8 `META` - metadata

UTF-8 JSON object. Free-form. Tools MAY write `{"src": "...", "gen": "..."}`. Readers MUST ignore unknown keys.

The `META` chunk is informational. It MUST NOT affect rendering. Tools that do not understand JSON MUST skip the chunk without error.

12.9 `THMB` - thumbnail

Raw PNG bytes. Used by file managers for previews. A renderer MUST NOT include `THMB` in the drawing. Tools that do not understand PNG MUST skip `THMB` without error.

The thumbnail SHOULD be at most 512x512. Readers MAY ignore it.

Chapter 13

Coordinate System

The canvas origin is `(0, 0)` at the top-left. X grows to the right, Y grows downward. This matches SVG, Canvas 2D, and most 2D graphics APIs.

Transforms are affine matrices:

```
| a  c  e |
| b  d  f |
| 0  0  1 |
```

A point `(x, y)` transforms to `(a*x + c*y + e, b*x + d*y + f)`.

Nested transforms compose from parent to child: the child's transform is applied first, then the parent's. In practice, a renderer multiplies matrices top-down.

13.1 Path arcs

`Arc` follows the SVG parameterization:

- `rx`, `ry` - radii of the ellipse,
- `rot` - rotation of the ellipse in radians,
- `large_arc` - 0 or 1,
- `sweep` - 0 or 1,
- `(x, y)` - end point.

If `rx == 0` or `ry == 0`, the arc is treated as a straight line to `(x, y)`. If the radii are too small to connect the current point to `(x, y)`, they are scaled up proportionally.

13.2 Stroke geometry

A stroke is centered on the path. `stroke_width` is the full width, so the visible band extends `stroke_width / 2` on each side. `cap` and `join` apply to the ends of subpaths and to corners.

`miter_limit` is the maximum ratio of miter length to stroke width. If the miter would exceed the limit, the join falls back to `bevel`.

Chapter 14

Error Model

All errors are fatal for the chunk or file being processed. Recommended error categories:

Category	When
format error	bad magic, truncated header, truncated chunk table, bad field
version error	`version` greater than implemented
codec error	unknown codec, codec not supported, decode failure
chunk error	malformed chunk payload, out-of-bounds offset

Implementations SHOULD distinguish these categories in their API, because callers may want to recover (skip a chunk) or fail (abort).

14.1 When to fail

A reader MUST fail with a format error when:

- the file is shorter than 32 bytes,
- `magic != b'BVG\x00'`,
- the chunk table is truncated,
- a chunk's `offset + size` exceeds the chunk data area.

A reader MUST fail with a version error when:

- `version > 1`.

A reader MUST fail with a codec error when:

- `codec_id` is not in `{0, 1, 2, 3, 4}`,
- the required codec is not supported,
- decoding fails (bad stream, truncated).

A reader MUST fail with a chunk error when:

- a chunk payload is shorter than its declared count implies,
- an index into `COLR`, `GRAD`, `IMGS`, `STYL` is out of range,
- a `float32` is NaN or infinite where the field requires a finite number.

14.2 When to recover

A reader SHOULD recover when:

- an unknown chunk type is encountered - skip it,
- `total_uncompressed` does not match the sum of decoded chunks - trust the decoded chunks, warn, continue,
- a reserved flag bit is set - ignore it, warn.

Recovery is a policy decision. Tools that open untrusted files SHOULD recover; tools that validate a file for conformance SHOULD fail.

Chapter 15

Change Log

- **1.0.0** - initial specification.

Chapter 16

Glossary

- **Affine matrix** - 2x3 matrix describing translation, rotation, scale, shear.
- **Chunk** - named, typed, compressed block of data.
- **Chunk table** - list of chunk records at the start of the data area.
- **Codec** - compression algorithm.
- **DEFLATE** - compression algorithm used by zlib.
- **DOM** - Document Object Model, a tree representation of XML.
- **Gradient** - smooth color transition.
- **LZ4** - fast lossless compression algorithm.
- **Palette** - list of RGBA colors referenced by index.
- **Pre-order** - tree traversal: parent, then children left to right.
- **RLE** - run-length encoding.
- **Scene tree** - the `NODE` chunk as a tree of groups and leaves.
- **Style** - fill, stroke, opacity, dash, blend mode.
- **SVG** - XML-based vector graphics format.
- **Zstandard** - fast lossless compression algorithm.
- **zlib** - DEFLATE with a zlib wrapper.

Chapter 17

References

- RFC 2119 - Key words for use in RFCs to Indicate Requirement Levels.
- SVG 2 - W3C Recommendation (for the arc parameterization).
- Zstandard - RFC 8878.
- LZ4 - https://github.com/lz4/lz4/blob/dev/doc/lz4_Frame_format.md
- zlib - RFC 1950 (ZLIB), RFC 1951 (DEFLATE).